

FUNDEMENTALS OF DATA SCIENCE

DSA0404

1. Student Average (student_average.py)

```
import numpy as np

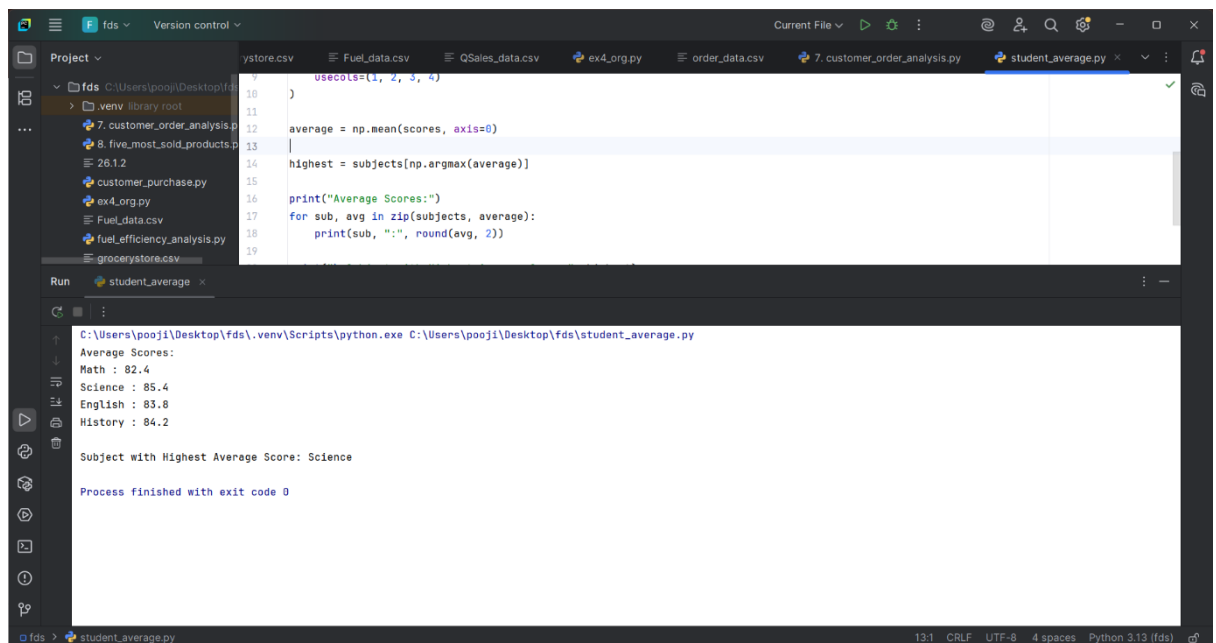
student_scores = np.loadtxt("Students_data.csv", delimiter=",", skiprows=1,
                             usecols=(1,2,3,4))

subjects = ["Math", "Science", "English", "History"]

average_scores = np.mean(student_scores, axis=0)

highest_subject = subjects[np.argmax(average_scores)]

print("Average scores:", np.round(average_scores, 2))
print("Subject with highest average score:", highest_subject)
```



```
Project: fds
  ystore.csv
  Fuel_data.csv
  QSales_data.csv
  ex4_org.py
  order_data.csv
  7_customer_order_analysis.py
  student_average.py
  7_customer_order_analysis.py
  8_five_most_sold_products.py
  26.1.2
  customer_purchase.py
  ex4_org.py
  Fuel_data.csv
  fuel_efficiency_analysis.py
  grocerystore.csv

Run: student_average
C:\Users\pooji\Desktop\fds\.venv\Scripts\python.exe C:\Users\pooji\Desktop\fds\student_average.py
Average Scores:
Math : 82.4
Science : 85.4
English : 83.8
History : 84.2
Subject with Highest Average Score: Science
Process finished with exit code 0
```

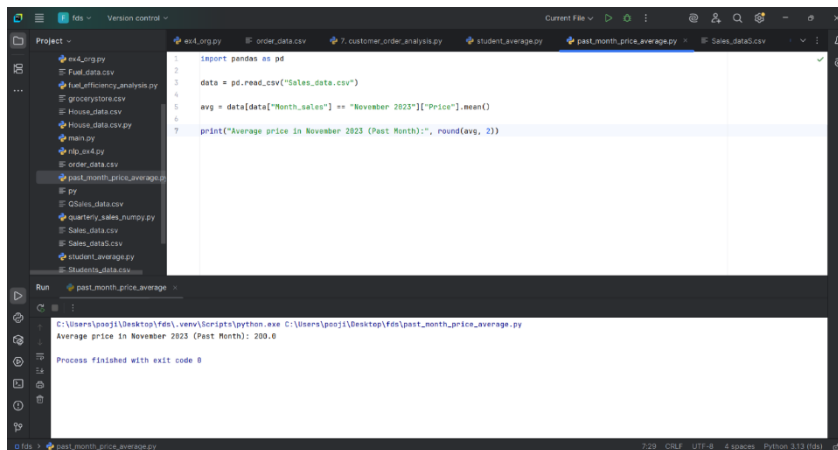
2. Past Month Price Average

```
import pandas as pd

df = pd.read_csv("Sales_data.csv")

average_price = df[df["Month_sales"] == "November 2023"]["Price"].mean()

print("Average price in November 2023 (Past Month):", round(average_price, 2))
```



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor in the center. The code editor contains the following Python code:

```
1 import pandas as pd
2 data = pd.read_csv("Sales_data.csv")
3 avg = data[data["Month_sales"] == "November 2023"]["Price"].mean()
4
5 print("Average price in November 2023 (Past Month):", round(avg, 2))
```

The Run button is highlighted, and the output console at the bottom shows the following output:

```
C:\Users\pooj\I\Desktop\fsd\venv\Scripts\python.exe C:\Users\pooj\I\Desktop\fsd\past_month_price_average.py
Average price in November 2023 (Past Month): 200.6
Process finished with exit code 0
```

3. House Average Price (house_average_price.py)

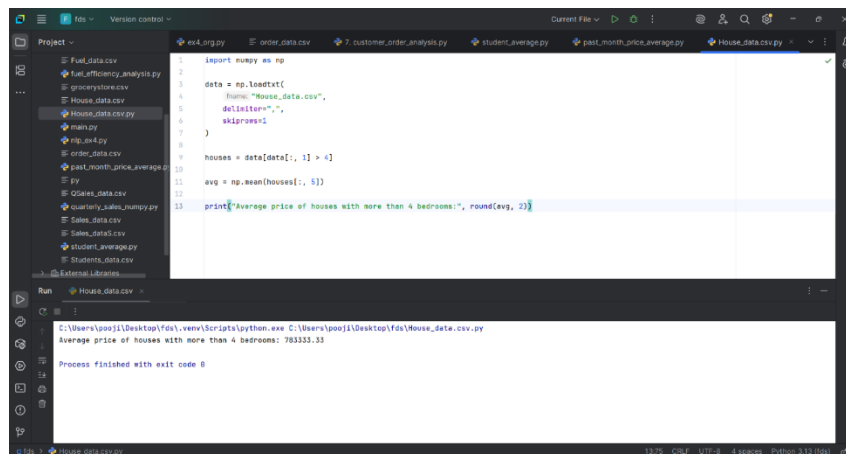
```
import numpy as np

house_data = np.loadtxt("House_data.csv", delimiter=",", skiprows=1)

houses_more_than_4 = house_data[house_data[:,1] > 4]

average_price = np.mean(houses_more_than_4[:,5])

print("Average price of houses with more than 4 bedrooms:", round(average_price, 2))
```



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor in the center. The code editor contains the following Python code:

```
1 import numpy as np
2 data = np.loadtxt(
3     filename="House_data.csv",
4     delimiter=",",
5     skiprows=1
6 )
7 houses = data[data[:, 1] > 4]
8
9 avg = np.mean(houses[:, 5])
10
11 print("Average price of houses with more than 4 bedrooms:", round(avg, 2))
```

The Run button is highlighted, and the output console at the bottom shows the following output:

```
C:\Users\pooj\I\Desktop\fsd\venv\Scripts\python.exe C:\Users\pooj\I\Desktop\fsd\House_data.csv.py
Average price of houses with more than 4 bedrooms: 78535.55
Process finished with exit code 0
```

4. Quarterly Sales (quarterly_sales_numpy.py)

```
import numpy as np

sales_data = np.genfromtxt("Sales_data.csv", delimiter=",", skip_header=1, dtype=str)

months = sales_data[:,1]

sales = sales_data[:,4].astype(float)

Q1 = 0.0

Q4 = 0.0

for i in range(len(months)):

    if ("January" in months[i]) or ("February" in months[i]) or ("March" in months[i]):

        Q1 += sales[i]

    elif ("October" in months[i]) or ("November" in months[i]) or ("December" in months[i]):

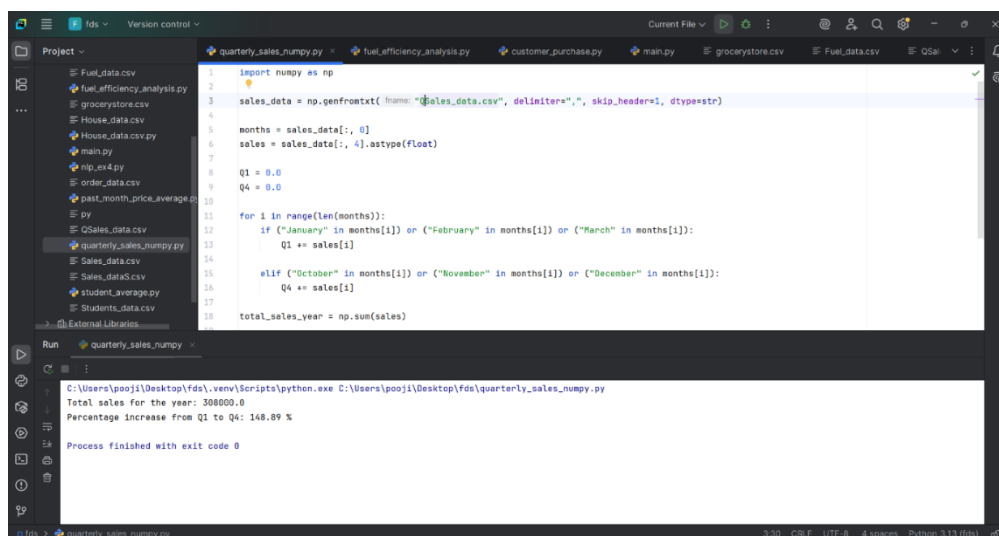
        Q4 += sales[i]

total_sales_year = np.sum(sales)

percentage_increase = ((Q4 - Q1) / Q1) * 100

print("Total sales for the year:", round(total_sales_year, 2))

print("Percentage increase from Q1 to Q4:", round((percentage_increase), 2), "%")
```



The screenshot displays a code editor with a project explorer on the left and a run console at the bottom. The project explorer lists various files, including 'quarterly_sales_numpy.py'. The code editor shows the Python script for calculating quarterly sales. The run console shows the output of the script, which is: 'Total sales for the year: 30800.0' and 'Percentage increase from Q1 to Q4: 148.09 %'. The process finished with exit code 0.

```
1 import numpy as np
2
3 sales_data = np.genfromtxt("Sales_data.csv", delimiter=",", skip_header=1, dtype=str)
4
5 months = sales_data[:, 1]
6 sales = sales_data[:, 4].astype(float)
7
8 Q1 = 0.0
9 Q4 = 0.0
10
11 for i in range(len(months)):
12     if ("January" in months[i]) or ("February" in months[i]) or ("March" in months[i]):
13         Q1 += sales[i]
14
15     elif ("October" in months[i]) or ("November" in months[i]) or ("December" in months[i]):
16         Q4 += sales[i]
17
18 total_sales_year = np.sum(sales)
19
20 percentage_increase = ((Q4 - Q1) / Q1) * 100
21
22 print("Total sales for the year:", round(total_sales_year, 2))
23
24 print("Percentage increase from Q1 to Q4:", round((percentage_increase), 2), "%")
```

Run: quarterly_sales_numpy.py

C:\Users\poorj\Desktop\fs\venv\scripts\python.exe C:\Users\poorj\Desktop\fs\quarterly_sales_numpy.py

Total sales for the year: 30800.0

Percentage increase from Q1 to Q4: 148.09 %

Process finished with exit code 0

5. Fuel Efficiency Analysis (fuel_efficiency_analysis.py)

```
import numpy as np

fuel_data = np.genfromtxt("Fuel_data.csv", delimiter=",", skip_header=1, dtype=str)

fuel_efficiency = fuel_data[:,7].astype(float)

make = fuel_data[:,8]

average_efficiency = np.mean(fuel_efficiency)

mazda_eff = fuel_efficiency[make == "mazda"]

audi_eff = fuel_efficiency[make == "audi"]

mazda_avg = np.mean(mazda_eff)

audi_avg = np.mean(audi_eff)

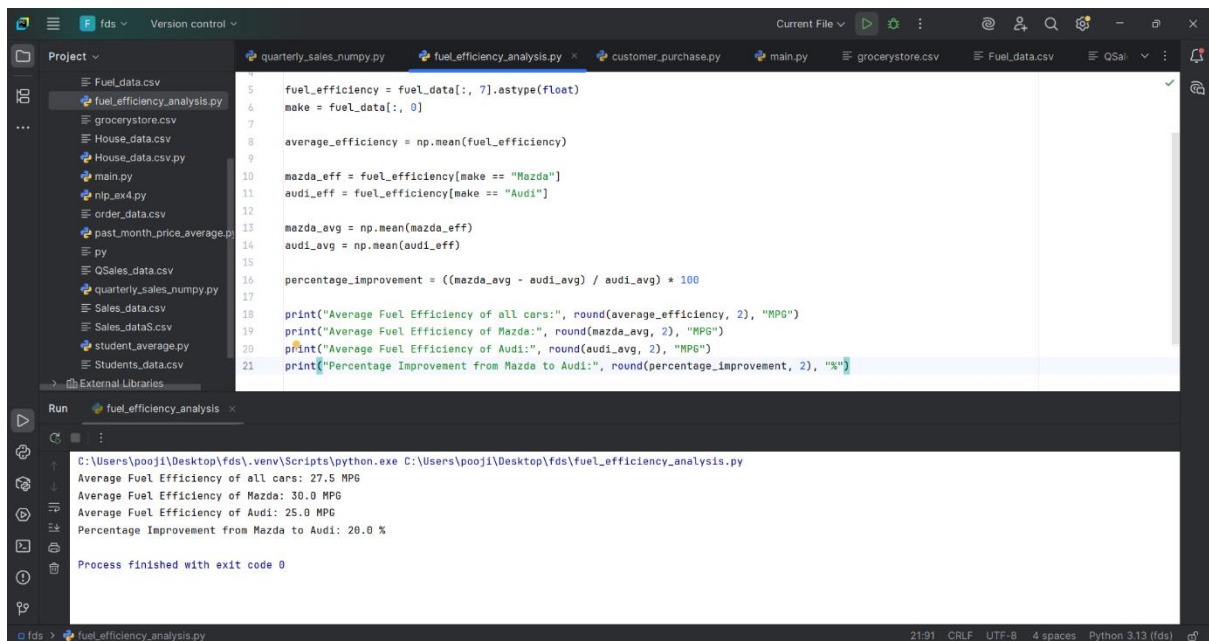
percentage_improvement = ((mazda_avg - audi_avg) / audi_avg) * 100

print("Average Fuel Efficiency of all cars:", round(average_efficiency, 2), "MPG")

print("Average Fuel Efficiency of Mazda:", round(mazda_avg, 2), "MPG")

print("Average Fuel Efficiency of Audi:", round(audi_avg, 2), "MPG")

print("Percentage Improvement from Mazda to Audi:", round(percentage_improvement, 2),
"%")
```



The screenshot shows a code editor with the file `fuel_efficiency_analysis.py` open. The code is as follows:

```
5 fuel_efficiency = fuel_data[:, 7].astype(float)
6 make = fuel_data[:, 8]
7
8 average_efficiency = np.mean(fuel_efficiency)
9
10 mazda_eff = fuel_efficiency[make == "Mazda"]
11 audi_eff = fuel_efficiency[make == "Audi"]
12
13 mazda_avg = np.mean(mazda_eff)
14 audi_avg = np.mean(audi_eff)
15
16 percentage_improvement = ((mazda_avg - audi_avg) / audi_avg) * 100
17
18 print("Average Fuel Efficiency of all cars:", round(average_efficiency, 2), "MPG")
19 print("Average Fuel Efficiency of Mazda:", round(mazda_avg, 2), "MPG")
20 print("Average Fuel Efficiency of Audi:", round(audi_avg, 2), "MPG")
21 print("Percentage Improvement from Mazda to Audi:", round(percentage_improvement, 2), "%")
```

The Run console shows the following output:

```
C:\Users\pooji\Desktop\fds\.venv\Scripts\python.exe C:\Users\pooji\Desktop\fds\fuel_efficiency_analysis.py
Average Fuel Efficiency of all cars: 27.5 MPG
Average Fuel Efficiency of Mazda: 30.0 MPG
Average Fuel Efficiency of Audi: 25.0 MPG
Percentage Improvement from Mazda to Audi: 20.0 %

Process finished with exit code 0
```

6. Customer Purchase (customer_purchase.py)

```
import pandas as pd

df = pd.read_csv("grocerystore.csv")

total_sales = df["Sales"].sum()

print("Total sales for the grocery store:", round(total_sales, 2))

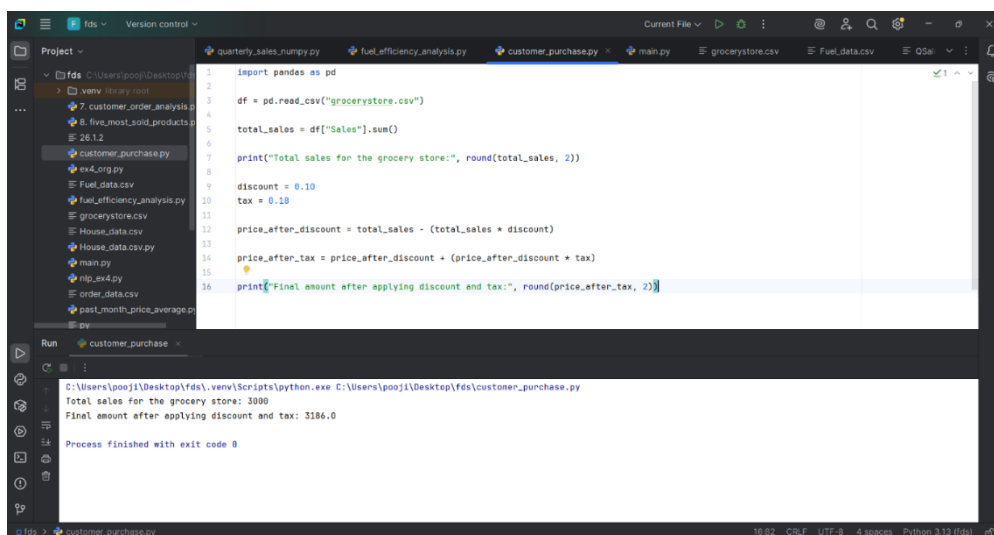
discount = 0.10

tax = 0.18

price_after_discount = total_sales - (total_sales * discount)

price_after_tax = price_after_discount + (price_after_discount * tax)

print("Final amount after applying discount and tax:", round(price_after_tax, 2))
```



```
1 import pandas as pd
2 df = pd.read_csv("grocerystore.csv")
3
4 total_sales = df["Sales"].sum()
5
6 print("Total sales for the grocery store:", round(total_sales, 2))
7
8 discount = 0.10
9 tax = 0.18
10
11 price_after_discount = total_sales - (total_sales * discount)
12
13 price_after_tax = price_after_discount + (price_after_discount * tax)
14
15 print("Final amount after applying discount and tax:", round(price_after_tax, 2))
16
```

Run customer_purchase.py

```
C:\Users\pooj\\Desktop\fds\venv\scripts\python.exe C:\Users\pooj\Desktop\fds\customer_purchase.py
Total sales for the grocery store: 3090
Final amount after applying discount and tax: 3186.0
Process finished with exit code 0
```

7. Customer Order Analysis (customer_order_analysis.py)

```
import pandas as pd

df = pd.read_csv("order_data.csv")

total_orders = df.groupby("Full Name")["Order Count"].sum()

avg_order_per_product = df.groupby("Items")["Order Total"].mean()

earliest_order = df["Order"].min()

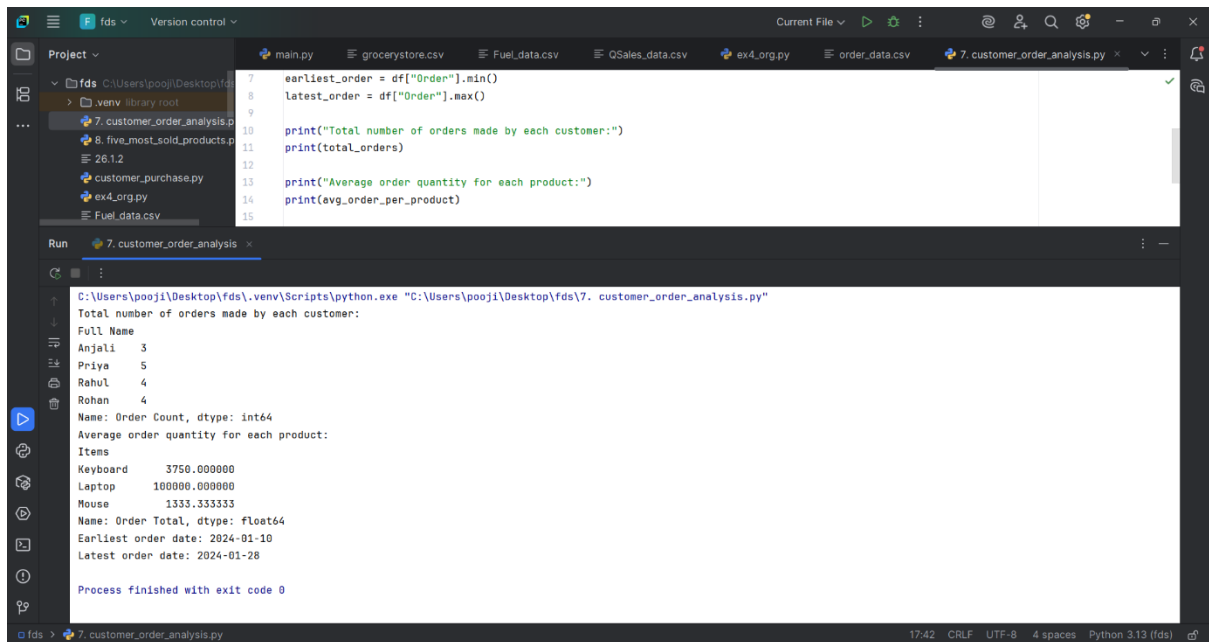
latest_order = df["Order"].max()

print("Total number of orders made by each customer:", total_orders)
```

```
print("Average order quantity for each product:", avg_order_per_product)
```

```
print("Earliest order date:", earliest_order)
```

```
print("Latest order date:", latest_order)
```



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The code editor displays a Python script for customer order analysis. The script calculates the earliest and latest order dates, the total number of orders per customer, the average order quantity per product, and the total order count. The output is displayed in the Run console at the bottom.

```
7 earliest_order = df["Order"].min()
8 latest_order = df["Order"].max()
9
10 print("Total number of orders made by each customer:")
11 print(total_orders)
12
13 print("Average order quantity for each product:")
14 print(avg_order_per_product)
15
```

Run console output:

```
C:\Users\pooji\Desktop\fds\venv\Scripts\python.exe "C:\Users\pooji\Desktop\fds\7. customer_order_analysis.py"
Total number of orders made by each customer:
Full Name
Anjali    3
Priya     5
Rahul     4
Rohan     4
Name: Order Count, dtype: int64
Average order quantity for each product:
Items
Keyboard    3750.000000
Laptop      100000.000000
Mouse       1333.333333
Name: Order Total, dtype: float64
Earliest order date: 2024-01-10
Latest order date: 2024-01-28
Process finished with exit code 0
```

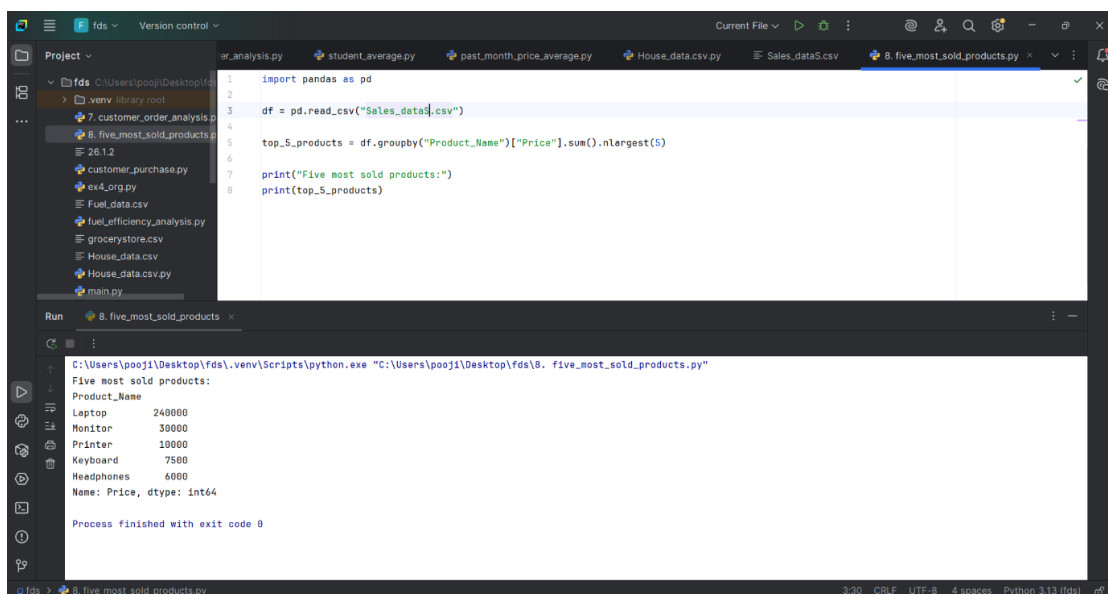
8. Five Most Sold Products (five most sold products.py)

```
import pandas as pd
```

```
df = pd.read_csv("Sales_data.csv")
```

```
top_5_products = df.groupby("Product_Name")["Price"].sum().nlargest(5)
```

```
print("Five most sold products:", top_5_products)
```



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The code editor displays a Python script for finding the five most sold products. The script imports pandas, reads the Sales_data.csv file, and uses groupby to find the top 5 products by total price. The output is displayed in the Run console at the bottom.

```
1 import pandas as pd
2
3 df = pd.read_csv("Sales_data.csv")
4
5 top_5_products = df.groupby("Product_Name")["Price"].sum().nlargest(5)
6
7 print("Five most sold products:")
8 print(top_5_products)
```

Run console output:

```
C:\Users\pooji\Desktop\fds\venv\Scripts\python.exe "C:\Users\pooji\Desktop\fds\8. five_most_sold_products.py"
Five most sold products:
Product_Name
Laptop      240000
Monitor     30000
Printer     10000
Keyboard    7500
Headphones   6000
Name: Price, dtype: int64
Process finished with exit code 0
```

9. Property Data Analysis (property_analysis.py)

```
import pandas as pd

property_data = pd.read_csv("property_data.csv")

avg_price = property_data.groupby("Location")["Listing Price"].mean()

more_than_4 = property_data[property_data["Bedrooms"] > 4]

largest_area = property_data.loc[property_data["Area"].idxmax()]

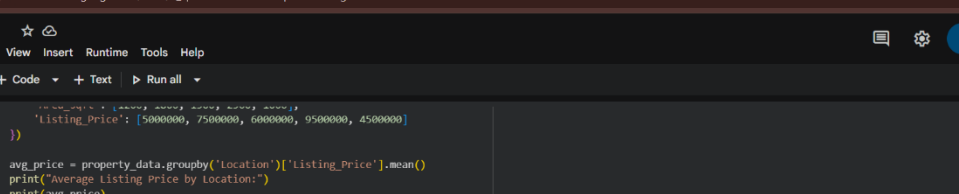
print("Average listing price in each location:")

print(avg_price)

print("Number of properties with more than four bedrooms:", len(more_than_4))

print("Property with the largest area:")

print(largest_area)
```



The screenshot shows the FDS IDE interface. The top bar includes a file explorer with 'Unfiled2.ipynb - Colab' and 'FDS - Colab'. The main workspace displays a Jupyter Notebook with the following code:

```
import pandas as pd
property_data = pd.read_csv('data/properties.csv')

# Task 1: Average Listing Price by Location
avg_price = property_data.groupby('Location')['Listing_Price'].mean()
print("Average Listing Price by Location:")
print(avg_price)

# Task 2: Properties with more than four bedrooms
count = property_data[property_data['Bedrooms'] > 4].shape[0]
print("\nProperties with more than four bedrooms:", count)

# Task 3: Property with Largest Area
largest_area = property_data.loc[property_data['Area_sqft'].idxmax()]
print("\nProperty with Largest Area:")
print(largest_area)
```

The output of the code is displayed below the code cells:

```
--- Average Listing Price by location:
Location
Bangalore    9500000.0
Chennai      5500000.0
Hyderabad    6000000.0
Name: Listing_Price, dtype: float64

Properties with more than four bedrooms: 2

Property with Largest Area:
Property ID    104
Location      Bangalore
Bedrooms       6
Area_sqft     2500
Listing_Price  9500000
Name: 3, dtype: object
```

10. Monthly Sales Line Plot (monthly_sales_line.py)

```
import matplotlib.pyplot as plt

months = ["Jan","Feb","Mar","Apr","May","Jun"]
sales = [120,150,180,170,200,220]

plt.plot(months, sales, marker="o")
```

```
plt.title("Monthly Sales Line Plot")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

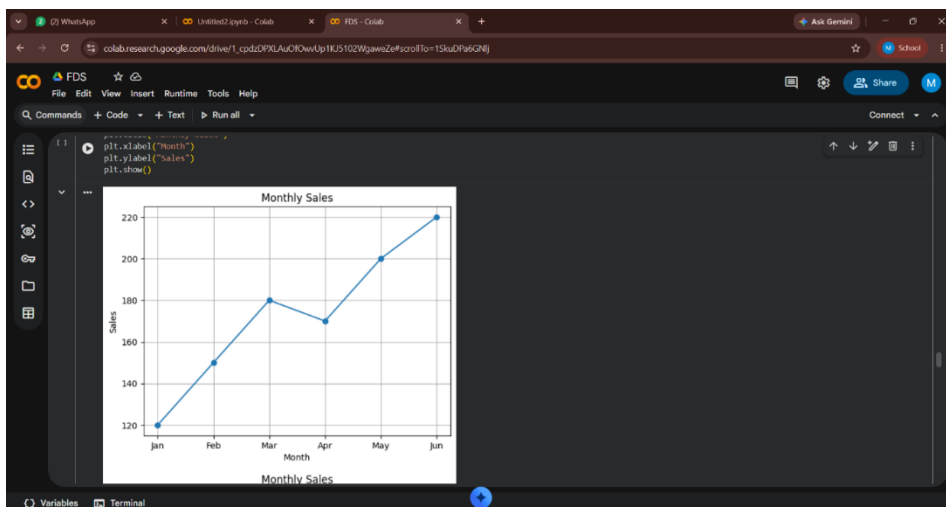
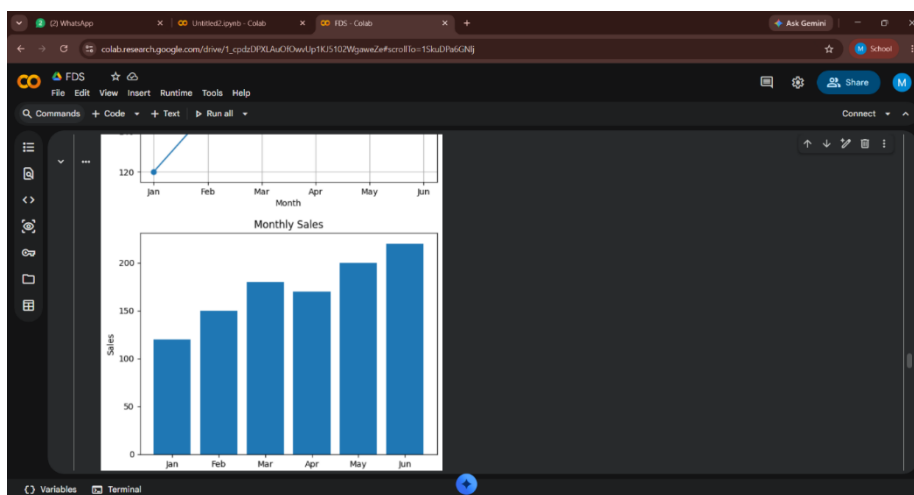
```
plt.bar(months, sales)
```

```
plt.title("Monthly Sales Bar Plot")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```



11. Sales Plots (sales_plots.py)

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [100, 120, 140, 160, 180, 200]

plt.plot(months, sales, marker="o")
plt.title("Sales Line Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.scatter(months, sales)
plt.title("Sales Scatter Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Sales Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```

